Binary Search Tree Data Structure

What is a Binary Search Tree?

A Binary Search Tree (BST) is a hierarchical data structure where each node has at most two children, referred to as the left child and right child. The key property of a BST is that for any node:

- All values in the left subtree are less than the node's value
- All values in the right subtree are greater than the node's value
- Both left and right subtrees are also binary search trees

BST Node Structure

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Basic BST Operations

1. Insertion

Inserting a new node while maintaining the BST property:

```java
public TreeNode insert(TreeNode root, int data) {
    // Base case: if tree is empty
    if (root == null) {
        return new TreeNode(data);
    }

    // If data is less than root, insert in left subtree
    if (data < root.data) {
        root.left = insert(root.left, data);
    }
    // If data is greater than root, insert in right subtree
    else if (data > root.data) {
        root.right = insert(root.right, data);
    }

    return root;
}
```

2. Search

Finding a specific value in the BST:

```java
public boolean search(TreeNode root, int key) {
    // Base case: root is null or key is found
    if (root == null) {
        return false;
    }
    if (root.data == key) {
        return true;
    }

    // Key is smaller than root's data, search left subtree
    if (key < root.data) {
        return search(root.left, key);
    }

    // Key is greater than root's data, search right subtree
    return search(root.right, key);
}
```

3. Deletion

Removing a node from the BST has three cases:

java

```java
public TreeNode delete(TreeNode root, int key) {
    if (root == null) {
        return root;
    }

    // Find the node to delete
    if (key < root.data) {
        root.left = delete(root.left, key);
    } else if (key > root.data) {
        root.right = delete(root.right, key);
    } else {
        // Node to be deleted found

        // Case 1: Node has no children (leaf node)
        if (root.left == null && root.right == null) {
            return null;
        }

        // Case 2: Node has one child
        if (root.left == null) {
            return root.right;
        }
        if (root.right == null) {
            return root.left;
        }

        // Case 3: Node has two children
        // Find inorder successor (smallest in right subtree)
        TreeNode successor = findMin(root.right);
        root.data = successor.data;
        root.right = delete(root.right, successor.data);
    }

    return root;
}

private TreeNode findMin(TreeNode root) {
    while (root.left != null) {
        root = root.left;
    }
    return root;
}
```

Tree Traversals

1. Inorder Traversal (Left → Root → Right)

```java
public void inorderTraversal(TreeNode root) {
    if (root != null) {
        inorderTraversal(root.left);
        System.out.print(root.data + " ");
        inorderTraversal(root.right);
    }
}
```

2. Preorder Traversal (Root → Left → Right)

```java
public void preorderTraversal(TreeNode root) {
    if (root != null) {
        System.out.print(root.data + " ");
        preorderTraversal(root.left);
        preorderTraversal(root.right);
    }
}
```

3. Postorder Traversal (Left → Right → Root)

```java
public void postorderTraversal(TreeNode root) {
    if (root != null) {
        postorderTraversal(root.left);
        postorderTraversal(root.right);
        System.out.print(root.data + " ");
    }
}
```

Additional BST Operations

Finding Minimum and Maximum Values

```java
// Find minimum value (leftmost node)
public int findMinimum(TreeNode root) {
    if (root == null) {
        throw new IllegalArgumentException("Tree is empty");
    }
    while (root.left != null) {
        root = root.left;
    }
    return root.data;
}

// Find maximum value (rightmost node)
public int findMaximum(TreeNode root) {
    if (root == null) {
        throw new IllegalArgumentException("Tree is empty");
    }
    while (root.right != null) {
        root = root.right;
    }
    return root.data;
}
```

Height of BST

```java
public int height(TreeNode root) {
    if (root == null) {
        return -1; // Height of empty tree is -1
    }

    int leftHeight = height(root.left);
    int rightHeight = height(root.right);

    return Math.max(leftHeight, rightHeight) + 1;
}
```

Validate BST

```java
public boolean isValidBST(TreeNode root) {
    return validate(root, Integer.MIN_VALUE, Integer.MAX_VALUE);
}

private boolean validate(TreeNode root, int min, int max) {
    if (root == null) {
        return true;
    }

    if (root.data <= min || root.data >= max) {
        return false;
    }

    return validate(root.left, min, root.data) &&
           validate(root.right, root.data, max);
}
```

Complete BST Implementation

java

```java
public class BinarySearchTree {
    private TreeNode root;

    public BinarySearchTree() {
        this.root = null;
    }

    public void insert(int data) {
        root = insert(root, data);
    }

    public boolean search(int key) {
        return search(root, key);
    }

    public void delete(int key) {
        root = delete(root, key);
    }

    public void inorderTraversal() {
        inorderTraversal(root);
        System.out.println();
    }

    // Include all the methods defined above

    public static void main(String[] args) {
        BinarySearchTree bst = new BinarySearchTree();

        // Insert elements
        bst.insert(50);
        bst.insert(30);
        bst.insert(70);
        bst.insert(20);
        bst.insert(40);
        bst.insert(60);
        bst.insert(80);

        // Display tree
        System.out.print("Inorder traversal: ");
        bst.inorderTraversal();

        // Search for elements
```

```java
        System.out.println("Search 40: " + bst.search(40));
        System.out.println("Search 90: " + bst.search(90));

        // Delete element
        bst.delete(30);
        System.out.print("After deleting 30: ");
        bst.inorderTraversal();
    }
}
```

Time and Space Complexity

Time Complexity:

- **Search**: O(h) where h is height of tree
  - Best case (balanced): O(log n)
  - Worst case (skewed): O(n)
- **Insertion**: O(h)
  - Best case: O(log n)
  - Worst case: O(n)
- **Deletion**: O(h)
  - Best case: O(log n)
  - Worst case: O(n)

Space Complexity:

- **Overall**: O(n) for storing n nodes
- **Recursive calls**: O(h) for stack space

Quiz Questions

1. What is the key property of a Binary Search Tree? a) All nodes have exactly two children b) Left subtree values < root < right subtree values c) The tree is always balanced d) Nodes are stored in ascending order

Answer: b) Left subtree values < root < right subtree values

2. What will be the result of inorder traversal of a BST? a) Random order b) Descending order c) Ascending order d) Level order

Answer: c) Ascending order

3. What is the time complexity of searching in a balanced BST? a) O(1) b) O(log n) c) O(n) d) $O(n^2)$

Answer: b) O(log n)

4. When deleting a node with two children in a BST, which node replaces it? a) Any leaf node b) The parent node c) Inorder successor or predecessor d) The root node

Answer: c) Inorder successor or predecessor

5. In the worst case, what does a BST degenerate into? a) A balanced tree b) A linked list c) A complete binary tree d) A heap

Answer: b) A linked list

Practice Coding Questions

1. Write a method to find the kth smallest element in a BST.

2. Implement a method to check if two BSTs are identical (same structure and values).

3. Write a function to find the Lowest Common Ancestor (LCA) of two nodes in a BST.

4. Create a method to convert a sorted array into a balanced BST.

5. Implement a function to find the distance between two nodes in a BST.